COMSATS University Islamabad, Lahore Campus

☑ Final Lab Exam Spring 2026

Course Title:	Parallel and Distributed Computing				Course Code:	CSC 334	Credit Hours: 3(2, 1)
Course Instructor:	Akhzar Nazir				Program Name:	BS SE	
Semester:	6th	Batch:	FA23	Section:	D	Date:	20/05/2026
Time Allowed:					**Maximum Marks:**		**100**
Student's Name:					Reg. No.		

Important Instructions / Guidelines:

- Lab exam is open book and open internet
- Good luck!

<CLO-5 - Creating> Develop an application using concepts of distributed and parallel computing in a team environment.

Adaptive Distributed Application Generation & Deployment System

Problem Statement

Modern distributed systems must dynamically adapt to available computational resources before executing large-scale workloads. You are required to design an **Adaptive Distributed Application Generation & Deployment System (ADAGDS)** that: Automatically analyzes system resources (CPU, GPU, memory, and remote worker capacity), determines feasible workload size, and generates multiple isolated application instances in parallel using a single codebase. The system must intelligently decide whether it can generate **20 applications or fewer or more**, based on real-time system capability.

System Overview

You are given:

- One shared modular codebase
- 10–20 JSON configuration files
- A distributed computing environment (optional multi-node setup)

Your system must:

- analyze system capacity
- decide number of apps to generate (≤ 20)
- distribute workloads across workers
- generate and deploy applications in parallel

Core System Requirements

1. Single Base Codebase Architecture

The system must have:

- one shared application engine
- modular architecture
- configuration-driven behavior
- template-based project scaffolding support

2. Configuration-Driven Multi-Application System

Each JSON configuration defines:

- application identity
- feature flags
- environment settings
- data source abstraction
- UI/theme metadata (abstract level)

Each configuration represents a **tenant specification**.

3. Distributed Job Execution System

The system must include:

- Master Node (Coordinator)
- Worker Nodes (Executors)

Responsibilities:

- Master distributes build jobs
- Workers generate application codebases independently
- Jobs run in parallel

4. Parallel Application Generation Engine

The system must:

- execute multiple builds concurrently
- support job queuing
- ensure safe parallel execution
- optimize resource usage

5. CRITICAL REQUIREMENT — Independent Codebase Generation (NEW / FIXED)

This is a core requirement:

Each generated application must be a FULLY INDEPENDENT PROJECT CODEBASE.

Output Requirement

If the system generates 2 applications:

```
/output
   /app1
   /app2
```

Each folder must contain:

✓ Complete standalone project:

- package.json
- node_modules setup capability
- src/
- public/ or mobile structure
- config files
- build scripts
- environment variables

✓ Independent execution:

Each app must run separately:

```
cd app1
npm install
npm start
cd app2
npm install
npm start
```

✓ No shared runtime dependency between apps

Each application is:

- isolated
- independently deployable
- independently buildable

6. Multi-Tenant Isolation Model

Each generated application must ensure:

- isolated configuration space
- no shared runtime state
- independent build output
- independent execution environment

7. Resource-Aware Scheduling System

Before execution:

- CPU analysis
- memory analysis
- optional GPU detection

- worker node capacity check

System must determine:

- how many apps can safely be generated
- optimal parallel execution level

8. Adaptive Workload Decision Engine

The system must decide dynamically:

- generate 20 apps (if resources allow)
- OR reduce workload (1–20 range)

9. Mandatory Minimum Execution Guarantee

Even under low resources:

The system must generate and successfully build at least 1–2 applications.

10. Deployment & Launch System

After generation:

- apps must be deployed to runtime environment
- each app gets a unique endpoint

Example:

```
app1 → http://server/app1
app2 → http://server/app2
```

At least 1–2 apps must be automatically launched.

11. Fault Tolerance System

System must handle:

- worker failure
- job retry
- partial build recovery
- no job loss

12. Logging & Monitoring System

Must track:

- build progress
- job execution time

- worker utilization
- system decisions (very important for grading)